

Acculturation docker

Harold Parpex

Sommaire

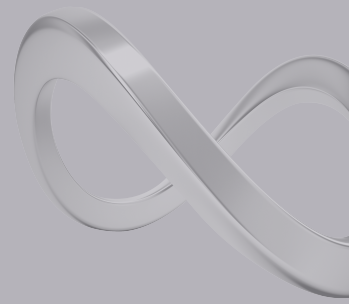


Objectifs

1. Pourquoi utiliser la conteneurisation?
2. VM vs Conteneur
3. Les concepts clés
4. Les cas d’utilisations
5. Les commandes
6. Les volumes et ports
7. Les layers
8. Démonstration de build
9. Démonstration de run
10. Les outils

Objectifs

- Comprendre le concept de conteneurisation
- Explorer les cas d’usage
- Comprendre l’architecture conteneur
- Apprendre l’utilisation de l’outil docker



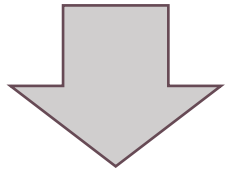
DevOps

Centre d’expertise d’accompagnement à la mise en place de la méthodologie DevOps de façon sécurisée. Ainsi que la mise en place d’outillage pour l’accélération du time to market.

1. Pourquoi utiliser la conteneurisation?

Problématique :

- Ça marche chez moi, mais pas ailleurs
- Besoin de portabilité, répétabilité et isolation



La **solution**, un environnement :

- Standardisé
- Partageable
- Normé
- Léger

2. VM vs Conteneur

Aspect	Machine virtuelle	Conteneur
Durée de démarrage	Plusieurs minutes	Quelque seconde
Taille	Plusieurs GO	Maximum quelque GO
Isolation	Très forte (OS séparé)	Noyau partagé
Performance	Moins bonnes (plus de couche)	Noyau partagé
Portabilité	Moyenne	Forte
Option de scalabilité	Faible	Forte

3. Les concepts clés

Images :
Une ***version figée*** d'une application. Partageable.
Exemple : Une image ubuntu + nginx

Instancie

Conteneur :
Une ***instanciation*** d'une image.
Image + exécution = conteneur

Créer

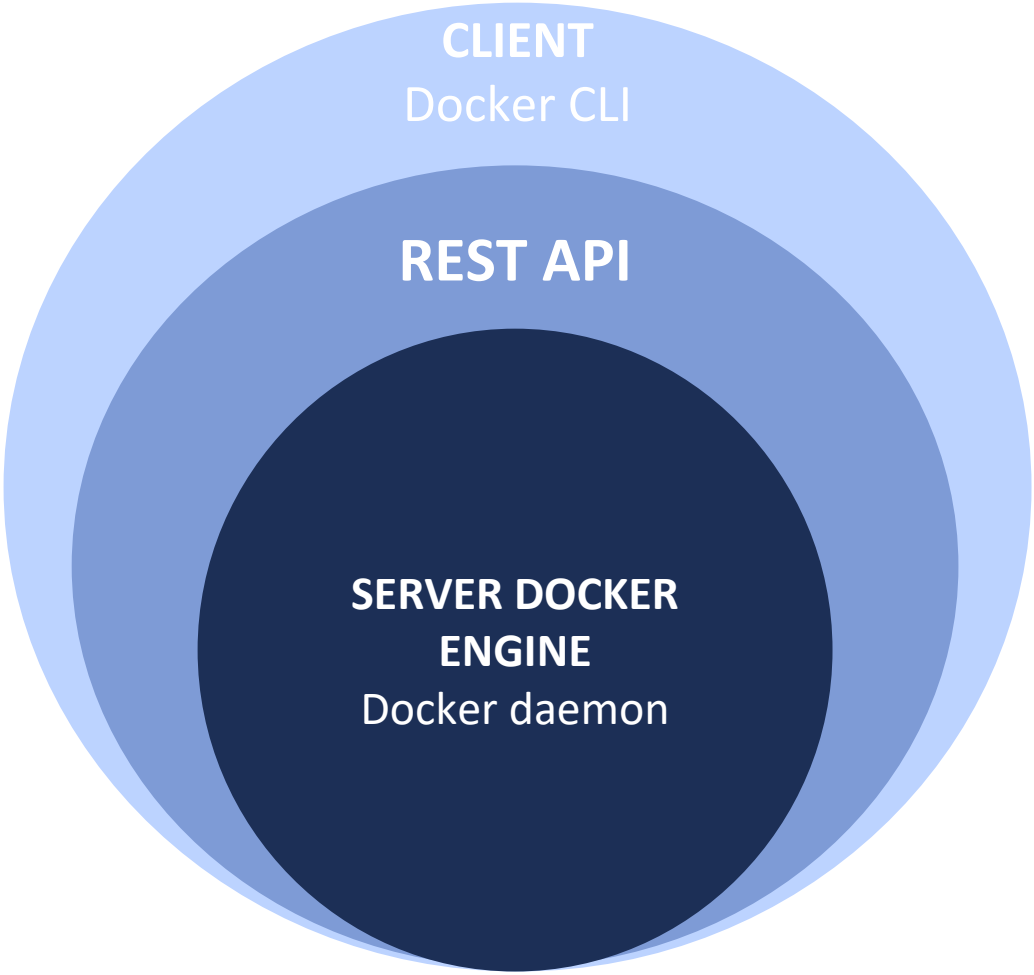
Stocke

Dockerfile :
Un fichier qui permet la ***création*** d'une image.
Exemple :

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "index.js"]
```

Registry :
Un outil qui sert au ***stockage*** et au ***partage*** des images
Exemple : dockerhub, ACR, Artifactory, Nexus

3. Les concepts clés



4. Les cas d’utilisations

💻 Développement local
Créer un environnement de dev reproductible sans rien installer en local (ex : Node.js, Python, base de données).

🔄 CI/CD automatisé
Utiliser Docker dans les pipelines pour construire, tester, livrer les apps dans un environnement identique à la prod.

🧪 Tests multi-environnements
Lancer des tests sur plusieurs versions d’un langage, d’un framework ou d’un OS (ex : Python 3.8, 3.9, 3.10).

⚙️ Microservices
Conteneuriser chaque microservice indépendamment, avec ses dépendances propres.

🚀 Déploiement rapide & stable
Déployer la même image en test, préprod et prod sans différences de comportement.

4. Les cas d’utilisations

- Scalabilité avec Kubernetes *(Docker + orchestration)*

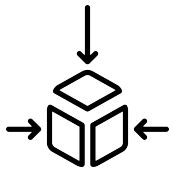
- Montée en charge automatique
  Kubernetes adapte le nombre de conteneurs selon la charge (auto-scaling horizontal).

- Haute disponibilité
  Redémarrage automatique en cas de crash, distribution sur plusieurs nœuds.

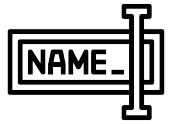
- Mise à jour continue (rolling update)
  Déployer des versions sans coupure, avec rollback possible en cas d'erreur.

- Gestion centralisée du réseau, des volumes, des secrets…

5. Les commandes



docker build –t mon-image . #le –t permet de nommer l'image, le . prend le Dockerfile dans le repertoire courant



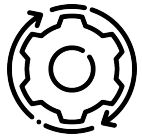
docker tag mon-image monregistry.com/monimage:latest #permet de renommer l'image ou de la formater pour l'envoyer sur un registry



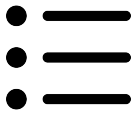
docker push monregistry.com/monimage:latest #envoie l'image sur un registry



docker pull registry.com/image:latest #Récupére une image sur un registry distant

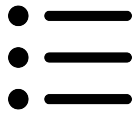


docker run monimage #Éxecute l'image en local

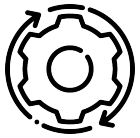


docker ps #Liste les conteneurs actifs

5. Les commandes



docker images #le –t permet de nommer l'image, le . prend le Dockerfile dans le répertoire courant



docker exec –it <id> bash #Accéde a un conteneur démarré

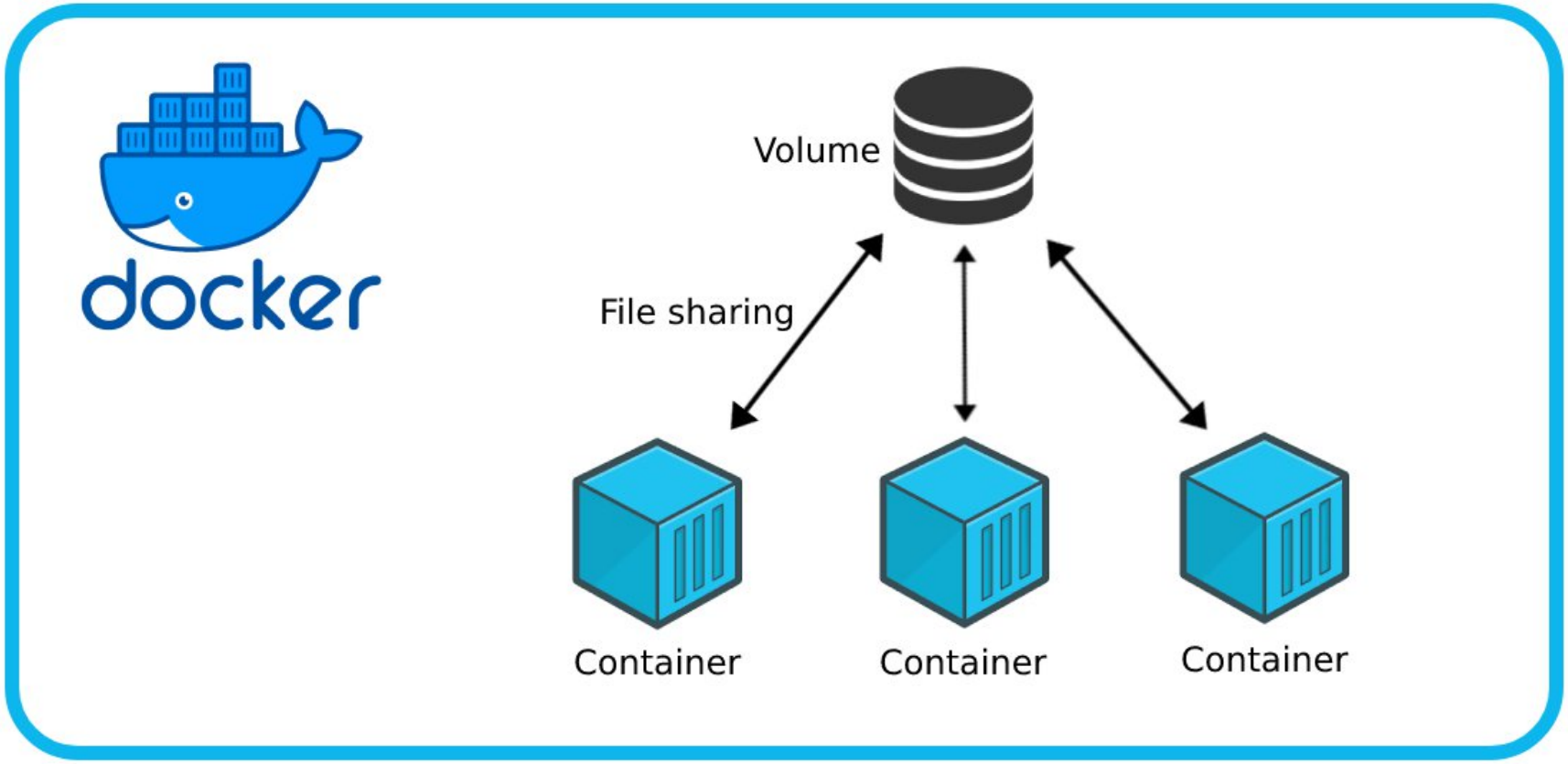


docker stop <id> #Arrete le conteneur



docker start <id> #Démarre le conteneur

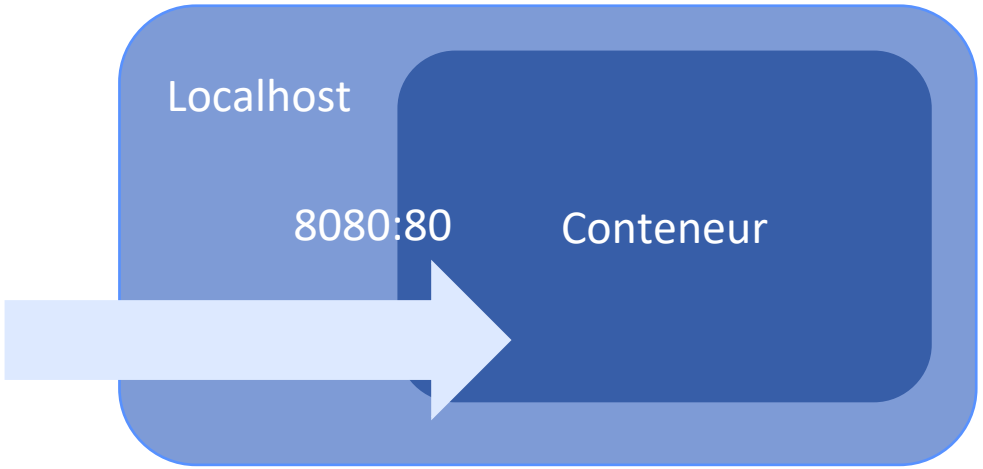
6. Les volumes et ports



Utilisation :
docker run --mount type=volume,src=<volume-name>,dst=<mount-path> monimage
docker run --volume <volume-name>:<mount-path> monimage
docker run –v <volume-name>:<mount-path> monimage

6. Les volumes et ports

Exemple pour un mapping de ports.
L'utilisateur effectuera une requête https://localhost:8080 pour appeler son conteneur qui est exécuter et en écoute sur le port 80.



Utilisation : docker run -p <port-host>:<port-conteneur> monimage

7. Les layers

Qu’est ce qu’un layer ?

- Une **image Docker** est construite par empilement de **couches (layers)**.
- Chaque **instruction dans un Dockerfile** (FROM, RUN, COPY, etc.) crée **une couche**.
- Ces couches sont **immuables** et **mises en cache** : si rien ne change, Docker ne les reconstruit pas.

Pourquoi utiliser des layers?

- On ne reconstruit que ce qui a changer (cache)
- Les layers sont réutilisables entre les différentes images
- Diminution des poids des images

7. Les layers



Dockerfile

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "index.js"]
```

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
```

```
#base Node.js (grosse couche)
#création du répertoire
#ajout des fichiers
#installation des dépendances
#ajout du reste de l'app
```

7. Les layers

Comment optimiser les Layers?

- Choisir une image de base légère
- Fusionner les commandes de run Exemple :

  RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*

  Au lieu de

  RUN apt-get update

  RUN apt-get install -y curl
- Copier les fichiers intelligement
- Utiliser un .dockerignore
- Utilisation du multistage Exemple :

```
FROM node:20 AS builder
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/index.js"]
```

8. Démonstration de build



9. Démonstration de run

10. Les outils

Un ensemble d’outils permettant de gérer du conteneur:

Portainer
Kubernetes
k9s
Docker Desktop
Podman
Openshift
Rancher
Codespace